

PyTorch로 배우는 딥러닝 4일 압축 과정

Day 1 - Part 1: 딥러닝 Why & How

4일 압축 과정 로드맵 (Day 1-2)

딥러닝 기초부터 CNN까지 - 체계적인 학습 여정

Day 1 (3.5시간) - 딥러닝 입문

- 딥러닝 Why & How

전통 ML vs 딥러닝 차이점, 계산 그래프

- 신경망 기초 이론

퍼셉트론, 활성화 함수, 역전파 알고리즘

- PyTorch 기초

Tensor, Dataset, DataLoader, 첫 분류 모델

Day 2 (9시간) - 딥러닝 심화

- DNN 구조 설계

은닉층, 뉴런 수, 배치 정규화, 드롭아웃

- 하이퍼파라미터 튜닝

학습률, 배치 크기, 옵티마이저 (Adam, SGD)

- 모델 평가와 검증

교차검증, 성능 지표, 과적합 방지

- CNN 기초

컨볼루션, 풀링, LeNet/VGG 구조

- 전이학습과 파인튜닝

사전 학습 모델 활용, torchvision.models

4일 압축 과정 로드맵 (Day 3-4)

시퀀스 모델부터 생성형 AI까지 - 최신 기술 마스터

Day 3 (9시간) - 시퀀스 & 생성 모델

- **RNN 기초**
RNN, LSTM, GRU 구조와 시퀀스 데이터 처리
- **자연어 처리 기초**
텍스트 전처리, 임베딩, 토큰나이제이션
- **Transformer 기본 개념**
Self-Attention, Multi-Head Attention
- **BERT 활용**
Huggingface로 BERT 파인튜닝
- **생성 모델 이론과 실습**
GAN, Diffusion 개념과 구현

Day 4 (9시간) - 생성형 AI 실전

- **생성형 AI와 LLM 이해**
최신 트렌드, 기존 AI와의 차이점
- **RAG 이론 및 실습**
Vector DB, 임베딩, 의미 기반 검색
- **AI 에이전트 설계**
프롬프트 엔지니어링, 챗봇 구현
- **LLM 파인튜닝**
Huggingface Transformers 활용

이번 파트의 학습 목표

이번 시간을 통해 우리는 다음을 이해하는 것을 목표로 합니다.

- 전통적인 머신러닝과 딥러닝의 **핵심적인 차이점**을 설명할 수 있습니다.
- 딥러닝 모델이 학습하는 기본 원리인 **계산 그래프**의 개념을 이해합니다.
- 모델의 성능에 영향을 미치는 **매개변수 수와 데이터 규모**의 관계를 파악합니다.

전통적 머신러닝 vs 딥러닝

데이터 처리와 학습 방식의 차이

전통적인 머신러닝과 딥러닝은 인공지능의 하위 분야이지만, 문제에 접근하는 방식에서 근본적인 차이를 보입니다.

전통적 머신러닝

- 전문가에 의한 특징 공학(**Feature Engineering**)이 필수적입니다.
- 주로 **정형 데이터**(표 형태의 데이터)에 강점을 보입니다.
- 모델의 구조가 비교적 단순하여 **해석이 용이**합니다.

딥러닝 (Deep Learning)

- 데이터로부터 **자동으로 특징을 학습**(**Feature Learning**)합니다.
- **비정형 데이터**(이미지, 텍스트, 음성) 처리에 매우 효과적입니다.
- 모델이 복잡하여 "**블랙박스**"로 비유되기도 합니다.

차이점 ①: 특징 공학 vs 특징 학습

누가 데이터의 '특징'을 찾아내는가?

- **특징 공학 (Feature Engineering)**

- 전통 ML에서 모델 학습 전, **사람(도메인 전문가)이 직접** 데이터에서 유용한 특징을 설계하고 추출하는 과정입니다.
- 예: 스팸 메일 분류 시, '특정 키워드 빈도', 'URL 포함 여부' 등을 직접 정의합니다.
- 모델 성능이 이 **수작업 특징의 품질에 크게 의존**합니다.

- **특징 학습 (Feature Learning)**

- 딥러닝 모델이 **내부의 여러 계층(Layer)을 통해 자동으로** 데이터의 특징을 추출하는 과정입니다.
- 초기 계층에서는 엣지, 색상 등 **단순한 패턴**을, 깊은 계층으로 갈수록 얼굴, 사물 등 **복잡하고 추상적인 패턴**을 학습합니다.
- 이 자동화된 특징 학습 능력이 딥러닝의 가장 큰 차별점입니다.

차이점 ②: 데이터량 및 확장성

얼마나 많은 데이터와 컴퓨팅 파워가 필요한가?

전통적 머신러닝

- 비교적 적은 데이터로도 준수한 성능을 낼 수 있습니다.
- 사람이 설계한 특징이 효과적이라면, 낮은 연산 자원(CPU)으로도 충분히 학습 가능합니다.
- 대규모의 복잡한 비정형 데이터에서는 성능의 한계가 명확합니다.

딥러닝 (Deep Learning)

- 일반적으로 매우 많은 양의 데이터를 필요로 합니다.
- 복잡한 모델 학습을 위해 고성능 하드웨어(GPU/TPU) 활용이 필수적입니다.
- 데이터가 많아질수록 모델 성능이 지속적으로 향상되는 경향을 보입니다.

차이점 ③: 모델 해석 가능성

모델의 예측 결과를 얼마나 설명할 수 있는가?

- **전통적 머신러닝 (White Box)**

- 의사결정나무, 선형 회귀 등은 모델의 구조가 단순하여 **결과를 설명하고 해석하기 용이**합니다.
- 선형 회귀의 가중치는 각 특징이 결과에 미치는 영향을, 의사결정나무는 결정 규칙을 명확히 보여줍니다.
- 예측의 근거가 중요한 **금융, 의료 분야**에서 중요한 장점입니다.

- **딥러닝 (Black Box)**

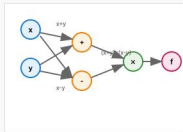
- 수많은 파라미터와 복잡한 계층 구조로 인해 "**블랙박스**"에 가깝다는 비판을 받습니다.
- 신경망이 특정 예측을 내놓은 이유를 직관적으로 설명하기 매우 어렵습니다.
- 이를 보완하기 위해 **SHAP**, **LIME** 등 **설명가능 AI(XAI)** 기술이 연구되고 있습니다.

딥러닝 학습 원리 ①: 계산 그래프

계산 그래프 (Computational Graph)란?

- 신경망의 모든 연산을 **노드(Node)**와 **엣지(Edge)**로 표현한 방향성 그래프입니다.
- 입력 데이터가 어떤 연산 과정을 거쳐 최종 출력에 도달하는지 그 흐름을 보여줍니다.
- **순전파(Forward Pass)**: 입력에서 출력 방향으로 계산을 수행하며 그래프를 생성합니다.
- **역전파(Backward Propagation)**: 출력에서 입력 방향으로 미분(기울기)을 계산합니다.
- 딥러닝 프레임워크(PyTorch 등)는 이 계산 그래프를 통해 **자동으로 미분 (autograd)**을 수행합니다.

예시: $f(x, y) = (x + y) \times (x - y)$



순전파: 입력 → 연산 → 출력
역전파: 기울기가 역방향으로 전파

딥러닝 학습 원리 ②: 순전파와 역전파

학습 과정: 순전파와 역전파의 반복

1. 순전파 (Forward Pass)

1. 입력 데이터를 모델에 통과시킵니다.
2. 각 계층에서 정의된 연산(행렬 곱, 활성화 함수 등)을 순차적으로 수행합니다.
3. 최종 예측값(Prediction)을 계산합니다.
4. 정답(Label)과 예측값의 차이인 **손실(Loss)**을 계산합니다.

이 모든 과정은 **계산 그래프**에 기록됩니다.

2. 역전파 (Backward Propagation)

1. 계산된 손실(Loss)로부터 시작합니다. (e.g., `loss.backward()`)
2. 계산 그래프를 거꾸로 거슬러 올라가며 **연쇄 법칙(Chain Rule)**에 따라 미분값을 계산합니다.
3. 각 파라미터(가중치, 편향)가 손실에 얼마나 영향을 미쳤는지(**기울기, Gradient**)를 구합니다.
4. 옵티마이저(예: SGD, Adam)가 이 기울기 정보를 사용해 **파라미터를 업데이트**합니다.

이 **순전파와 역전파 과정**을 수많은 데이터에 대해 반복(Epoch)하면서 모델은 점차 최적의 파라미터를 찾아갑니다.

성능의 열쇠: 파라미터, 데이터, 그리고 균형

모델 복잡도와 데이터 규모의 관계

- **과소적합 (Underfitting)**

- 모델이 너무 단순하여 (파라미터가 적음) 데이터의 패턴을 충분히 학습하지 못하는 상태입니다.
- 훈련 데이터에 대한 성능조차 낮게 나옵니다.

- **과적합 (Overfitting)**

- 모델이 너무 복잡하여 (파라미터가 많음) 훈련 데이터에만 과도하게 최적화되는 상태입니다.
- 훈련 데이터는 잘 맞추지만, 새로운 데이터(테스트 데이터)에 대한 성능은 떨어집니다. (일반화 실패)

- **좋은 모델 (Good Fit)**

- 모델의 복잡도와 데이터의 양이 **적절한 균형**을 이루어, 훈련 데이터의 패턴을 잘 학습하면서도 새로운 데이터에 대한 일반화 성능이 높은 상태입니다.

결론: 모델의 용량(Capacity)을 데이터 규모에 맞추는 것이 좋은 성능과 일반화 능력을 얻는 비결입니다.

Case Study: 모델과 데이터의 최적 비율은?

DeepMind의 Chinchilla 연구가 주는 교훈

"무조건 큰 모델이 정답은 아니다. **적절한 데이터로 잘 학습된, 알맞은 크기의 모델**이 더 우수할 수 있다."

GPT-3 (OpenAI)

- 파라미터 수: ~1,750억 개
- 학습 토큰 수: ~3,000억 개
- 분석: 모델 규모에 비해 데이터가 부족한 "**언더트레인(Under-trained)**" 상태로 평가됩니다.

Chinchilla (DeepMind)

- 파라미터 수: ~700억 개 (GPT-3보다 작음)
- 학습 토큰 수: ~1.4조 개 (GPT-3보다 4배 이상 많음)
- 결과: 더 작은 모델임에도 불구하고, **더 많은 데이터로 학습하여 GPT-3보다 높은 성능**을 달성했습니다.

Chinchilla 연구는 주어진 컴퓨팅 자원에서 최적의 성능을 내기 위해, **모델 파라미터 1개당 약 20개의 학습 토큰**이 이상적인 비율임을 제시했습니다.

딥러닝이 현대 AI에서 효과적인 이유

딥러닝은 왜 강력한가?

오늘날 이미지, 음성, 자연어 처리 등 복잡한 AI 과제에서 딥러닝이 비약적인 성능 향상을 이룬 핵심 이유는 다음과 같습니다.

1. 복잡한 비선형 패턴 학습 능력

- 수많은 파라미터를 통해 데이터 속의 미묘하고 복잡한 고차원 구조를 포착할 수 있습니다.

2. 계층적 특징 학습 (Hierarchical Feature Learning)

- 단순한 특징(선, 색)에서 복잡한 특징(형태, 객체)으로 점차 추상화된 개념을 학습합니다.

3. 대규모 데이터 활용 및 확장성

- 빅데이터와 GPU 등 병렬 연산 하드웨어의 발전에 힘입어 데이터가 많을수록 성능이 향상됩니다.

4. 엔드투엔드 (End-to-End) 학습

- 입력부터 출력까지 전체 과정을 하나의 모델로 통합하여 최적화하므로, 중간 단계의 오류 개입을 최소화합니다.

다양한 분야를 혁신하는 딥러닝

컴퓨터 비전

CNN 기반 모델이 이미지 분류, 객체 탐지에서 인간을 뛰어넘는 성능을 보입니다.

- 의료 영상 분석 (종양 식별)
- 자율주행 자동차 (보행자, 표지판 인식)

자연어 처리 (NLP)

Transformer 기반 언어 모델(BERT, GPT)이 언어의 문맥과 의미를 이해하고 생성합니다.

- 기계 번역, 챗봇, 감성 분석
- 검색 엔진, 질의응답 시스템

음성 인식

End-to-End 딥러닝 모델이 사람의 말을 높은 정확도로 텍스트로 변환합니다.

- 스마트폰 음성 비서 (Siri 등)
- 콜센터 자동 응답 시스템

이번 시간 정리

- **전통 ML vs. 딥러닝**: 가장 큰 차이는 **특징 공학의 자동화 여부**에 있으며, 이로 인해 데이터 요구량, 성능, 해석 가능성 등에서 차이가 발생합니다.
- **딥러닝 학습 원리**: **계산 그래프**를 기반으로 **순전파**(예측 및 손실 계산)와 **역전파**(기울기 계산 및 파라미터 업데이트)를 반복하며 학습합니다.
- **성능과 균형**: 모델 성능은 **모델의 복잡도(파라미터)**와 **데이터의 양** 사이의 균형이 중요하며, 과적합/과소적합을 피해야 합니다.
- **딥러닝의 강점**: 복잡한 패턴 학습, 계층적 표현, 대규모 데이터 활용, End-to-End 학습 능력 덕분에 현대 AI의 핵심 기술로 자리 잡았습니다.

질문 있으신가요?
